

define methods that are declared in the interfaces:

```
type Shape interface {
    area() float64
}

/* define a circle */
type Circle struct {
    x,y,radius float64
}

/* define a rectangle */
type Rectangle struct {
    width, height float64
}

/* define a method for circle for implementation of Shape.
area()*/
func(cir Circle) area() float64 {
    return math.Pi * cir.radius * cir.radius
}

/* define a method for rectangle for implementation of Shape.
area()*/
func(rec Rectangle) area() float64 {
    return rec.width * rec.height
}
```

Functions

A function is a collection of statements to perform a specific task. Every Go program must have at least one function, which is `main()`. A function is defined as follows:

```
func fun_name( [param list] ) [return_types]
{
    // body of the function
}
```

func: The `func` keyword is required to declare it as a function.

Func name: The name of the function.

Parameters: Used to store the value that is being passed to the function. A function may or may not contain parameters.

Return type: The list of data types of the values the function returns.

Function body: The function body contains a collection of statements that define the behaviour of the function.

Here is an example. This function is used to find the maximum number between two numbers. Here, 'Max function' contains two parameters and, after processing, it will return the maximum number as a result.

```
package main

import "fmt"
```



Figure 4: Max function

```
func main() {
    /* local variable definition */
    var num1 int = 10
    var num2 int = 20
    var result int

    /* calling a function to get max value */
    result = max(num1, num2)

    fmt.Printf("Maximum value is : %d\n", result )
}

/* function returning the max between two numbers */
func max(num1, num2 int) int {
    /* local variable declaration */
    var result int

    if (num1 > num2) {
        result = num1
    } else {
        result = num2
    }
    return result
}
```

Pointers

A pointer stores the address of the memory's location. Before using pointers in a program, we have to declare them. The syntax for pointer declaration is as follows:

```
var pointer_name *pointer_data_type
```

Here, `pointer_data_type` is the pointer's base type. It can be any valid Go data type. The asterisk is used to declare a variable as a pointer. Valid pointer declarations are:

```
var p *int           /* pointer to an integer */
var f *float32       /* pointer to a float */
```

The actual data type for all pointers is the same, as pointers will only store the memory's address. The difference between pointers of different data types is the data type of the variable the pointer points to.

The '*' and '&' operators

'*' is used to find the value pointer variables. By dereferencing a pointer, we can access the value the pointer points to. So for '*x = 0', we are storing 0 in the memory location x refers to.